

Base Game Start

I used the 'Make Cookie Clicker in 10 Minutes (Unity Tutorial)' by Sasquatch B Studios to build the base project.

You can watch the video if you'd like, I only followed up to about the 8-minute mark to ensure the project remains simple and manageable for the students.

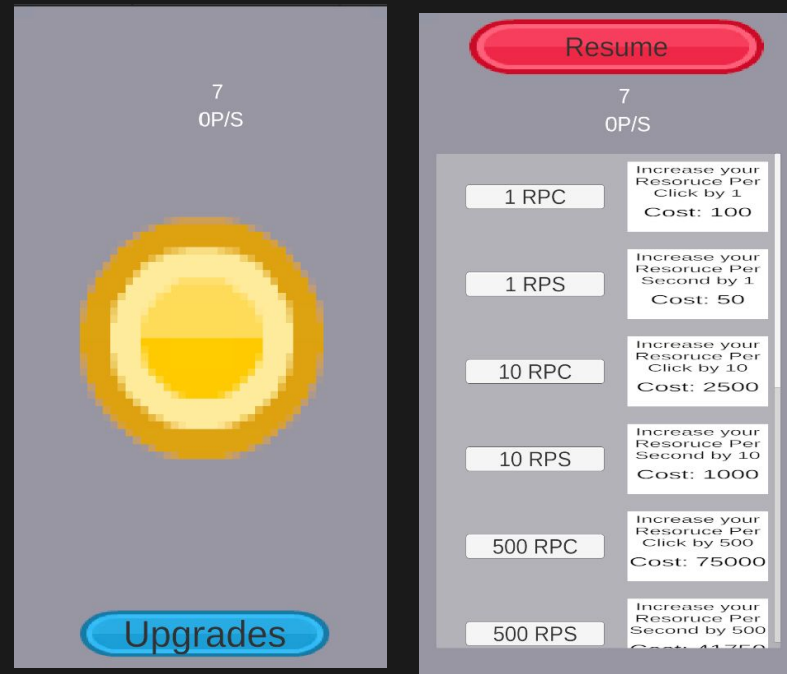


Game Overview

Here is an overview of the two-screen layout setup for the project:

Screen Structure

- **Main Game Screen:** Displays the current resource count, the resources-per-second rate, the clickable object, and a button to navigate to the Upgrades Screen.
- **Upgrades Screen:** Features dynamically instantiated upgrade buttons loaded at launch, allowing players to purchase upgrades that increase their resources per click.

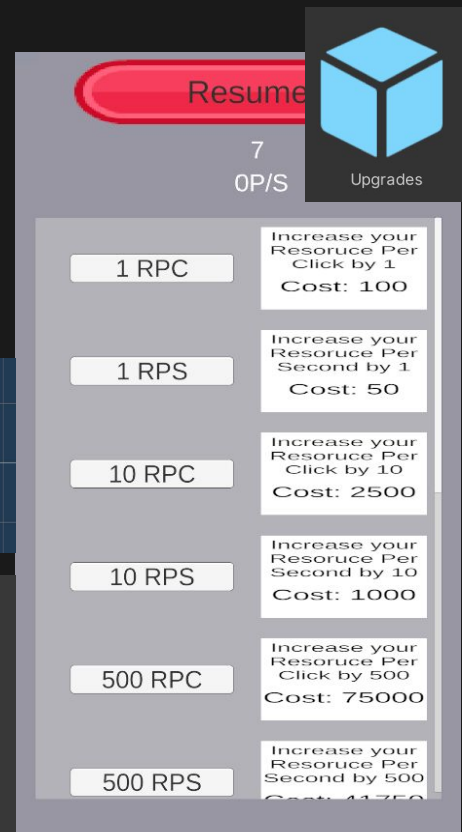
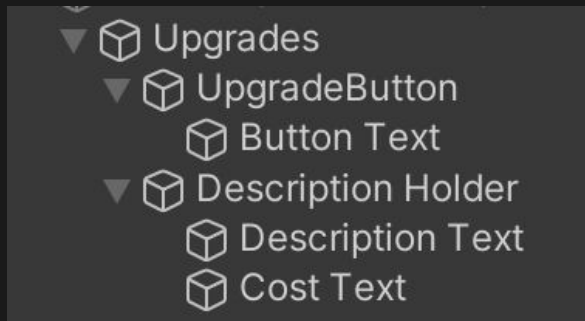
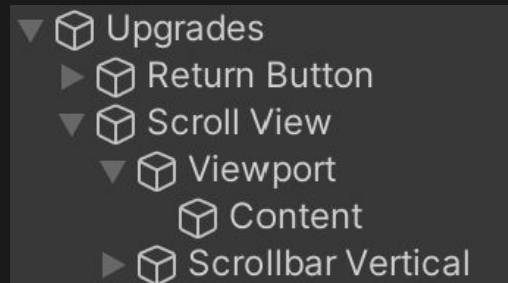


Upgrade Screen & Upgrade Buttons

Upgrade Screen & Button Mechanics

- **UI Hierarchy:** The Upgrade Screen utilizes a **Scroll View**, which contains a **Viewport** housing the **Content** container. All upgrade buttons are populated inside this Content container.
- **Dynamic Instantiation:** Upgrade buttons are not present in the scene by default. They are dynamically instantiated at runtime during initialization by the Game Manager script.
- **Button Components:** Each instantiated button item consists of three core components:
 - **Interactive Button:** Handles the player's click action to purchase the upgrade.
 - **Description Text:** Displays details regarding the upgrade type and stats.
 - **Cost Text:** Displays the current resource cost required to purchase the upgrade.

All three components and their respective text/data values are dynamically assigned when the game launches.

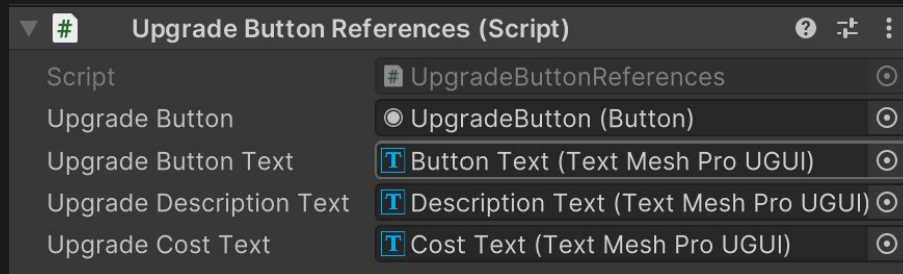
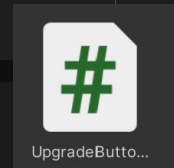


UpgradeButtonReferences

Upgrade Button References Script Setup

- **UpgradeButtonReferences Script:** Acts as a reference holder attached to each upgrade button prefab. It stores connections to all dynamic UI components:
 - **Button:** The interactive UI button element.
 - **Button Text:** Label text on the button itself.
 - **Description Text:** Info text displaying the upgrade details.
 - **Cost Text:** Display text showing the current purchase cost.
- **Initialization:** During Awake / game start, the Game Manager reads data from the assigned Scriptable Object and binds these UI component references dynamically.

```
// Holds direct references to elements inside an Upgrade Button prefab.  
// This allows other scripts to easily edit the button text without using expensive searching methods like GetComponentInChildren.  
# 1 asset usage 3 usages  
public class UpgradeButtonReferences : MonoBehaviour  
{  
    public Button UpgradeButton; # UpgradeButton (Button)  
    public TextMeshProUGUI UpgradeButtonText; # Button Text (Text Mesh Pro UGUI)  
    public TextMeshProUGUI UpgradeDescriptionText; # Description Text (Text Mesh Pro UGUI)  
    public TextMeshProUGUI UpgradeCostText; # Cost Text (Text Mesh Pro UGUI)  
}
```



Game Manager

Here is an breakdown of the Game Manager structure, which controls the main game logic, handles button actions, and updates the UI views:

Game Manager Architecture

- **Initialization (Awake):** Sets up core references, component connections, and dynamically instantiates all upgrade buttons at launch.
- **Click Actions & Animations:** Handles resource collection and triggers visual animations when the player clicks the central button.
- **Update View:** Updates and refreshes all on-screen text elements (such as current resource totals and generation rates).
- **Navigation Buttons:** Controls scene and panel transitions between the main game screen and the upgrade screen.
- **Direct Resource Adjustments:** Provides helper functions invoked by upgrade buttons to modify resource counts directly.
- **Purchase Upgrades:** Checks whether the player has sufficient resources to buy an upgrade, applies the purchase, and updates the upgrade cost based on predefined button settings.

GameManager

```
// Awake runs before Start when the scene loads
// Event function
private void Awake()
{
    // Enforce the Singleton pattern so there is only ever one GameManager
    if (instance == null) {
        instance = this;
    }

    // Refresh text displays on screen launch
    UpdateResourceUI();
    UpdateResourcePerSecond();

    // Ensure player starts on the main game screen, not the upgrade menu
    _upgradeCanvas.SetActive(false);
    MainGameCanvas.SetActive(true);

    // Fetch the component and create all upgrade buttons on startup
    _initializeUpgrades = GetComponent<InitializeUpgrades>();
    _initializeUpgrades.Initialize(ResourceUpgrades, _upgradeUIToSpawn, _upgradeUIParent);
}
```

Click Actions and Animations

Update View

Navigation Buttons

Direct Resource Adjustments

Purchasing Upgrades



GameMan...

Resource Upgrade

Resource Upgrade Architecture

- **ResourceUpgrade (Abstract Base Class):** Defines fundamental upgrade properties, including base cost, cost scaling/increments, and upgrade efficiency. It contains the abstract ApplyUpgrade method that must be implemented by child classes.
- **ResourceUpgradePerClick and ResourceUpgradePerSecond** Inherits from ResourceUpgrade and implements ApplyUpgrade.

These implemented ApplyUpgrade methods are invoked by the respective upgrade buttons whenever a player purchases an upgrade.

```
// Abstract base class representing a generic upgrade.
// Uses ScriptableObject so individual upgrades can be created as data assets in Unity project files.
🔗 No asset usages  📦 5 usages  📦 2 inheritors
public abstract class ResourceUpgrade : ScriptableObject
{
    public float UpgradeAmount;  🔗 Changed in 0+ assets

    public double OriginalUpgradeCost = 100;  🔗 Changed in 0+ assets
    public double CurrentUpgradeCost = 100;  🔗 Changed in 0+ assets
    public double CostIncreaseMultiplierPerPurchase = 0.05f; // Cost increases by 5% each purchase  🔗 Chan

    public string UpgradeButtonText;  🔗 Changed in 0+ assets
    [TextArea(3, 10)] // Gives multi-line text input field inside Unity Inspector
    public string UpgradeButtonDescription;  🔗 Changed in 0+ assets

    // Abstract method: Every child class must define what actually happens when purchased
    📦 1 usage  📦 2 overrides
    public abstract void ApplyUpgrade();

    // Runs automatically in the Unity Editor whenever values are changed in the Inspector
    🔗 Event function
    private void OnValidate() {
        CurrentUpgradeCost = OriginalUpgradeCost;
    }
}
```



ResourceUpgrade

ResourceUpgradePerClick & ResourceUpgradePerSecondPerClick

Derived Upgrade Classes

- **ResourceUpgradePerClick:** Inherits from ResourceUpgrade and implements the specific action triggered when its upgrade button is clicked, increasing the resource gain per click.
- **ResourceUpgradePerSecondPerClick:** Inherits from ResourceUpgrade and implements the action triggered when its upgrade button is clicked, increasing the passive per-second resource generation rate.

Both classes override the base ApplyUpgrade method to execute their respective upgrade logic upon purchase.

```
// A specific type of upgrade that inherits from ResourceUpgrade.  
// Allows creating assets directly from Unity menu: Create -> Resource Upgrade -> Resource Per Click  
[CreateAssetMenu(menuName = "Resource Upgrade/Resource Per Click", fileName = "Resource Per Click")]  
# 6 asset usages  
public class ResourceUpgradePerClick : ResourceUpgrade  
{  
    // Implementation of abstract method: Adds value to manual click strength  
    # 0+1 usages  
    public override void ApplyUpgrade() {  
        GameManager.instance.ResourcePerClickUpgrade += UpgradeAmount;  
    }  
}
```



ResourceUpgradePerClick



ResourceUpgradePerSecondPerClick

```
// A specific type of upgrade that adds passive income per second over time.  
// Allows creating assets directly from Unity menu: Create -> Resource Upgrade -> Resource Per Second  
[CreateAssetMenu(menuName = "Resource Upgrade/Resource Per Second", fileName = "Resource Per Second")]  
# 6 asset usages  
public class ResourceUpgradePerSecondPerClick : ResourceUpgrade  
{  
    // Implementation of abstract method: Spawns a timer object and updates total passive income  
    # 0+1 usages  
    public override void ApplyUpgrade() {  
        // Spawns (instantiates) a timer prefab into the game scene at position (0,0,0)  
        GameObject go = Instantiate(GameManager.instance.ResourcePerSecondObjectToSpawn, Vector3.zero, Quaternion.identity);  
  
        // Configures the spawned timer object with this upgrade's resource amount  
        go.GetComponent<ResourcePerSecondTimer>().ResourcePerSecond = UpgradeAmount;  
  
        // Updates GameManager's passive resource tracking and UI display  
        GameManager.instance.SimpleResourcePerSecondIncrease(UpgradeAmount);  
    }  
}
```

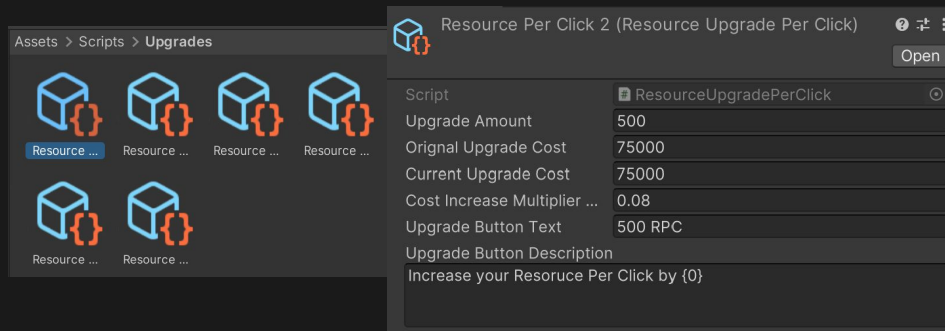
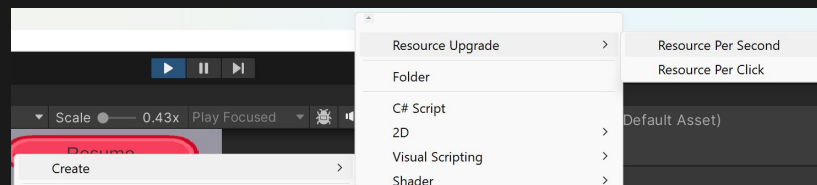
Scriptable Object

Scriptable Object Setup

- **ResourceUpgrade (Scriptable Object):** Serves as a data container for predefined upgrade values (such as costs and increment amounts) that can be passed directly into Button Prefabs.
- **CreateAssetMenu Attribute:** Derived classes like ResourceUpgradePerClick utilize CreateAssetMenu to allow easy creation of new upgrade assets directly within the Unity Editor.
- **Pre-configured Assets:** Located in the Script Upgrade folder, you will find 12 pre-configured Scriptable Objects with preloaded values representing different scales of upgrades.

```
// Abstract base class representing a generic upgrade.  
// Uses ScriptableObject so individual upgrades can be created as data assets in Unity project files.  
# No asset usages 5 usages 2 inheritors  
public abstract class ResourceUpgrade : ScriptableObject
```

```
// A specific type of upgrade that adds passive income per second over time.  
// Allows creating assets directly from Unity menu: Create -> Resource Upgrade -> Resource Per Second  
[CreateAssetMenu(menuName = "Resource Upgrade/Resource Per Second", fileName = "Resource Per Second")]  
# 6 asset usages  
public class ResourceUpgradePerSecondPerClick : ResourceUpgrade
```



Initialize Upgrades

Initialize Upgrades Functionality

- **InitializeUpgrades Script / Algorithm:** Iterates through all Scriptable Objects stored in the Resource Upgrades array within the Game Manager.
- **Dynamic Instantiation:** For each Scriptable Object in the array, it spawns a corresponding button prefab inside the Upgrade Screen's Content container and assigns its initial values and UI references.

```
// Dynamically creates UI buttons in the upgrade menu based on data objects
// 1 asset usage 2 usages
public class InitializeUpgrades : MonoBehaviour
{
    // Takes an array of upgrade data, creates a button for each, and configures them
    // 1 usage
    public void Initialize(ResourceUpgrade[] upgrades, GameObject UIToSpawn, Transform spawnParent)
    {
        for (int i = 0; i < upgrades.Length; i++)
        {
            int currentIndex = i;

            // Instantiate (clone) a UI button prefab into the designated layout container
            GameObject go = Instantiate(UIToSpawn, spawnParent);

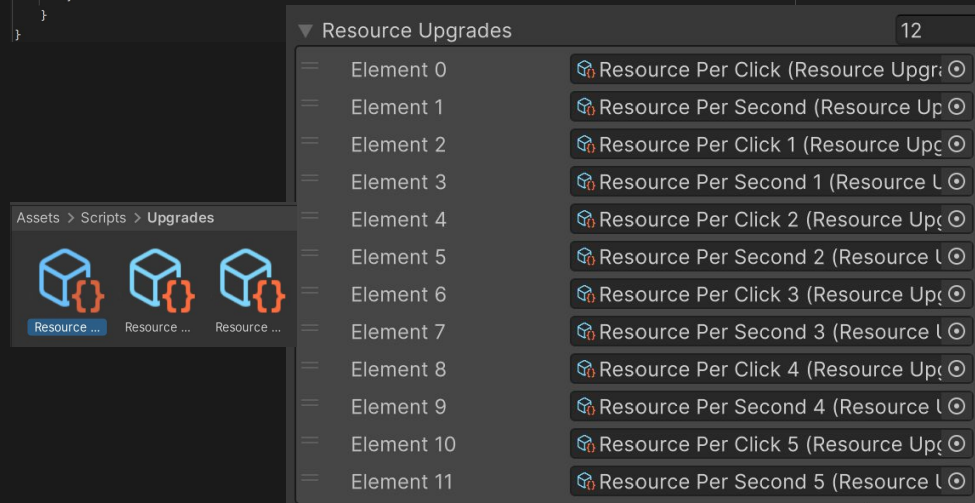
            // Reset current cost back to initial starting cost
            upgrades[currentIndex].CurrentUpgradeCost = upgrades[currentIndex].OriginalUpgradeCost;

            // Grab component references on the newly created button to update its text
            UpgradeButtonReferences buttonRef = go.GetComponent<UpgradeButtonReferences>();
            buttonRef.UpgradeButtonText.text = upgrades[currentIndex].UpgradeButtonText;
            buttonRef.UpgradeDescriptionText.SetText(upgrades[currentIndex].UpgradeButtonDescription, upgrades[currentIndex].UpgradeAmount);
            buttonRef.UpgradeCostText.text = "Cost: " + upgrades[currentIndex].CurrentUpgradeCost;

            // Assign a click listener so clicking the button calls GameManager purchase logic
            buttonRef.UpgradeButton.onClick.AddListener(delegate { GameManager.instance.OnUpgradeButtonClick(upgrades[currentIndex], buttonRef); });
        }
    }
}
```



InitializeUpgra..



Resource Per Second

Resource Per Second Timer Setup

- **ResourcePerSecond Timer Mechanics:** Runs an internal timer loop that tracks elapsed time during gameplay.
- **Resource Generation:** Every time the timer completes its cycle, it automatically increases the total resource count based on the current resources-per-second rate and resets the timer.



ResourcePerSecond

```
// Handles passive income generation over time
1 asset usage 1 usage

public class ResourcePerSecondTimer : MonoBehaviour {
    public float TimerDuration = 1f; // Runs every 1 second by default 1 Unchanged

    2 Frequently called 2 usages
    public double ResourcePerSecond { get; set; }

    private float _counter;

    // Update is called once per frame
    3 Event function
    private void Update() {
        // Accumulate time passed since last frame
        _counter += Time.deltaTime;

        // When accumulated time reaches duration target, award resources and reset timer
        4 if (_counter >= TimerDuration) {
            GameManager.instance.SimpleResourceIncrease(ResourcePerSecond);
            _counter = 0;
        }
    }
}
```



ResourceP...

Alpha Threshold

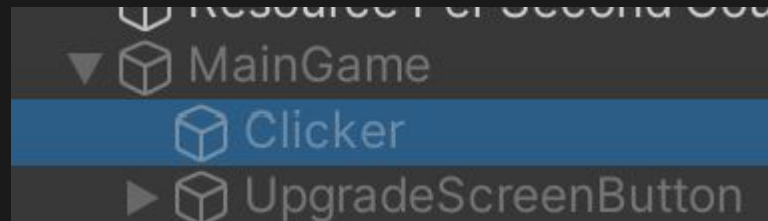
Alpha Threshold Script Mechanics

- **AlphaThreshold Script:** Restricts click detection so the button only reacts when a non-transparent pixel is clicked.
- **Problem Solved:** Standard UI images in Unity use rectangular bounding boxes, which triggers clicks even when tapping empty/transparent corners. This script ensures only the visible graphic receives clicks.
- **Placement:** Attached directly to the main clicker button object.

```
public class AlphaThreshold : MonoBehaviour
{
    private Image _resourceImage;

    ✦ Event function
    private void Awake() {
        _resourceImage = GetComponent<Image>();

        // Prevents fully transparent parts of an image from triggering a button click event
        _resourceImage.alphaHitTestMinimumThreshold = 0.001f;
    }
}
```



Judging Criteria

Category	Description	1 Point (Below Expectations)	3 Points (Meets Expectations)	5 Points (Exceeds Expectations)
Theme Alignment	To Be Announced	Shows little to no connection to the theme, or the implementation feels entirely incidental.	Clearly incorporates the theme into core concept or presentation.	Integrates the theme seamlessly into mechanics and design in a surprising or insightful way.
Innovative Mechanics	Integration and novelty of new code interactions. AI may assist with code snippets, but must not build the game for the creator.	Heavy reliance on full AI-generated code or standard pre-built systems with minimal personal understanding or original logic.	Uses custom code with AI assistance for snippets. Mechanics integrate well into the core gameplay loop.	Complex and highly custom code built from scratch or utilizing AI to enhance the gameplay experience. Implemented by student and not Agent.
Design	Quality of level/User Interface layouts and pacing. Custom-built spaces rank higher than premade environment templates.	Uses unmodified premade template maps or AI-built layouts that feel empty, broken, or uninspired.	Levels are thoughtfully constructed using custom or adapted assets to create balanced, functional spaces.	Masterful, original level design that builds unique environments to progressively test and highlight mechanics.
Presentation	Distinct visual and audio identity. Strictly no AI art allowed. Custom art ranks highest, followed by store/site assets, with AI art penalized.	Contains AI-generated art, or uses an uncoordinated mix of stock assets that hurts visual clarity.	Uses store or stock assets effectively, well-fitted to a cohesive, clear aesthetic with sound design.	Features entirely original visual and audio art, or heavy, creative modification of store assets for a unique identity.

Possible Questions

Theme Alignment

1. How does your core gameplay mechanic directly reflect or connect to the announced theme?
2. What was your thought process behind incorporating the theme into the game's concept?

Innovative Mechanics

1. What specific custom logic or code did you write yourself versus relying on pre-built scripts or AI assistance?
2. How does your primary new mechanic alter or improve the player's core gameplay experience?

Design

1. How do your level layouts guide the player to learn and test the game's primary mechanics?
2. Did you design the levels and environments from scratch, or how did you adapt existing assets to fit your layout?

Presentation

1. Where did you source your art and audio assets, and did you create or heavily modify any of them yourself?
2. How did you ensure a consistent visual and audio style throughout the game while avoiding AI-generated art?